

Python libraries for machine learning

Library	Purpose	Key features	Basic code example
NumPy	Numerical computing	Multi-dimensional arrays, vectorized operations, linear algebra	<code>import numpy as nparr = np.array([1, 2, 3])</code>
Pandas	Data manipulation and analysis	DataFrames, handling missing data, group-by filtering	<code>import pandas as pddf = pd.read_csv("data.csv")</code>
Matplotlib	Data visualisation	Line plots, histograms, scatter plots	<code>import matplotlib.pyplot as pltplt.plot(x, y)</code>
Seaborn	Statistical visualisations	Heatmaps, box plots, pair plots	<code>import seaborn as snsns.boxplot(data=df)</code>
scikit-learn	Classical machine learning	Model training, cross-validation, pipelines, metrics	<code>from sklearn.linear_model import LinearRegressionmodel = LinearRegression().fit(X, y)</code>
Tensor-Flow	Deep learning	Neural networks, distributed training, deployment tools	<code>import tensorflow as tfmodel = tf.keras. Sequential([...])</code>
Keras	High-level deep learning API (via TF)	Easy model building, prototyping	<code>from tensorflow import keraskeras.layers.Dense(...)</code>
PyTorch	Deep learning and research	Dynamic computation graph, GPU acceleration	<code>import torchx = torch.tensor([1.0, 2.0])</code>
OpenCV	Computer vision	Image processing, object detection	<code>import cv2img = cv2.imread('image.jpg')</code>
spaCy	Natural language processing	Tokenization, POS tagging, named entity recognition	<code>import spacynlp = spacy.load("en_core_web_sm")</code>
XGBoost	Gradient boosting	High-performance ML for structured data	<code>import xgboost as xgbmodel = xgb.XGBClassifier(). fit(X, y)</code>

is the conversion of categorical variables into a numerical format because most machine learning algorithms require a numerical input. The scaling and normalisation of numerical features improve model performance, especially for algorithms sensitive to these features.

```
from sklearn.model_selection
import train_test_split from sklearn.preprocessing
import StandardScaler
Splitting data into features and target
X = df.drop('target', axis=1) y = df['target']
Splitting into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2)
Scaling the features
scaler = StandardScaler() X_train_scaled = scaler.fit_
```

```
transform(X_train) X_test_scaled = scaler.transform(X_test)
```

Model selection

Having preprocessed the data, it is now time to select the correct model for the task at hand. Model selection generally depends on the problem and its corresponding dataset. Machine learning problems can be broadly categorised into two: supervised, where labels are available on the data; and unsupervised, where the data does not have any labels. For supervised learning tasks, the model could be as simple as a linear model like logistic regression, or it could be more complex involving support vector machines (SVMs) or decision-trees.

```
from sklearn.linear_model import LogisticRegression
Initializing the model
model = LogisticRegression()
```